

hey chat my DSBDA practical exam is there on the corner help me to prepare according to each practical lets start

Alright, good—you're starting *before* the exam panic kicks in, that's already a smart move.

Let's do this properly: we'll go **practical-by-practical**, not random theory. I'll help you with:

- what usually comes in that practical
- step-by-step commands/code
- common viva questions
- expected outputs
- shortcuts to save time in exam

First Step — Tell Me Your Practical List

DSBDA (Data Science & Big Data Analytics) practicals vary by college, but usually include things like:

- Data preprocessing (handling missing values, normalization)
- Data visualization (matplotlib / seaborn)
- Correlation & regression
- Classification (Naive Bayes / Decision Tree)
- Clustering (K-Means)
- Association rule mining (Apriori)
- Hadoop / MapReduce basics (sometimes)
- Python / Pandas operations

👉 I don't want to assume and teach wrong stuff.

Send me your exact practical list (photo or text)

OR tell me:

“start with practical 1”

How We'll Prepare (Exam-Oriented)

For each practical, I'll give you:

1. ♦ Aim (what to write in journal)
 2. ♦ Theory (6–8 mark ready)
 3. ♦ Code (clean + exam-friendly)
 4. ♦ Output explanation
 5. ♦ Viva questions (VERY important)
 6. ♦ Common mistakes (to avoid losing marks)
-

⚡ Small Reality Check

Don't try to “understand everything deeply” right now.

Focus on:

- Writing correct code fast
- Explaining it confidently
- Handling viva

That's how you score.

👉 Send your **Practical 1** and we'll start immediately.



see i will give you problem statement and output of code i already practiced from them tell me what code is important only for that practical and explain or teach me all the concepts related to it so that i would not fail in viva !!

Good—that's actually the *best* way to prepare for a practical exam. You've already done the practice, now we'll make sure you can **defend it in viva + write it fast under pressure**.

Here's how I'll help you for *each practical* you send:

🔥 What I'll Extract From Your Problem

When you give:

-  Problem statement

-  Output (or screenshot)

I'll give you:

1. **Minimum Required Code (Exam Version)**

- Only the **important lines** (no unnecessary fluff)
 - Clean, easy to remember
 - With small comments so you can recall logic
-

2. **Concepts You MUST Know (Viva Shield)**

I'll explain:

- Why that code is used
 - What each step does
 - Alternative questions examiner may twist
-

3. **Top Viva Questions (with answers)**

Like:

- “Why did you use this function?”
 - “What if data is missing?”
 - “Difference between normalization & standardization?”
 - etc.
-

4. **Common Mistakes (where students lose marks)**

- Wrong library import
 - Syntax errors
 - Misunderstanding output
 - Overexplaining / underexplaining
-

5. **Journal-Ready Content (if needed)**

- Aim
 - Theory (6–8 marks ready)
 - Conclusion
-

Important Rule (for you)

Don't just memorize code.

In viva, they'll attack like this:

"What happens if I remove this line?"

"Why not use another method?"

"Explain output in simple words."

I'll train you exactly for that.

Let's Start

Send me:

 **Practical 1 problem statement + output**

Even if it's messy or screenshot text, send it—I'll clean it and break it down.

We'll go one-by-one until you're fully exam ready.



pract_1_merged.pdf
PDF

Perfect. This is exactly what I needed 

You've already done the practical—now I'll make you **viva-proof + exam-ready**.

I've carefully read your file  `pract_1_merged` and your problem statement. Let's break **Practical 1: Data Wrangling** properly.



PRACTICAL 1 – DATA WRANGLING (FULL PREP)



MINIMUM CODE YOU MUST REMEMBER (Exam Version)

Write **ONLY** this in exam (clean + sufficient):

<> Python



▶ Run

```
import pandas as pd
import numpy as np
from sklearn import preprocessing

# Load dataset
df = pd.read_csv("IRIS.csv")

# Basic info
print(df.shape)
print(df.info())
print(df.describe())

# Check missing values
print(df.isnull().sum())

# Data types
print(df.dtypes)

# Normalization (Min-Max Scaling)
scaler = preprocessing.MinMaxScaler()
num_cols = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
df[num_cols] = scaler.fit_transform(df[num_cols])

# Convert categorical to numerical
label = preprocessing.LabelEncoder()
df['species'] = label.fit_transform(df['species'])

print(df.head())
```

👉 This alone covers **80% marks**

CONCEPTS YOU MUST KNOW (VERY IMPORTANT FOR VIVA)

◆ 1. What is Data Wrangling?

👉 Answer:

Data wrangling is the process of:

- cleaning data
 - transforming data
 - preparing data for analysis
-

♦ 2. What is Pandas?

👉 Python library used for:

- data manipulation
 - handling datasets in tabular form (DataFrame)
-

♦ 3. What is DataFrame?

👉 Table-like structure with:

- rows = records
 - columns = features
-

♦ 4. df.shape

👉 Returns:

```
(150, 5)
```



- ✓ 150 rows
 - ✓ 5 columns
-

♦ 5. df.info() (VERY IMPORTANT)

From your output (page 2)  pract_1_merged :

- No null values
- 4 float columns
- 1 object column (species)

👉 Viva answer:

It gives summary including datatype, non-null count, memory usage

♦ 6. df.describe()

👉 Gives:

- mean
- std
- min, max
- quartiles

👉 Only for **numeric data**

◆ 7. Missing Values

Python



▶ Run

```
df.isnull().sum()
```

👉 Output = 0 → means **no missing values**

👉 Viva question:

Q: What if missing values exist?

👉 Answer:

- fill using mean/median
 - or drop rows
-

◆ 8. Data Types

Python



▶ Run

```
df.dtypes
```

👉 Output:

- float64 → numeric
 - object → categorical
-

◆ 9. NORMALIZATION (VERY IMPORTANT 🔥)

What you used:

Python



▶ Run

```
MinMaxScaler()
```

👉 Formula:

$$X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

👉 Output range:

✓ Between 0 and 1

👉 Why needed?

- to bring all features to same scale
- improves ML performance

👉 From your output (page 2):

values like:

```
0.2222, 0.6250...
```



◆ 10. Label Encoding

Python



Run

```
LabelEncoder()
```

👉 Converts:

```
Iris-setosa → 0  
Iris-versicolor → 1  
Iris-virginica → 2
```



👉 Used when:

- categorical variable exists

◆ 11. One-Hot Encoding (YOU ALSO USED)

Python



Run

```
OneHotEncoder()
```

👉 Converts:

```
0 → [1,0,0]  
1 → [0,1,0]  
2 → [0,0,1]
```



👉 Difference from Label Encoding:

Label Encoding

assigns numbers

may mislead model

One Hot Encoding

creates binary columns

safer for ML

◆ 12. df.head(), df.tail()

👉 Used to view:

- first 5 rows
- last 5 rows

◆ 13. df.loc vs df.iloc (EXAM FAVORITE)

df.loc → label-based

Python

```
df.loc[:, ["sepal_length", "sepal_width"]]
```



Run

df.iloc → index-based

Python

```
df.iloc[0:3, 0:2]
```



Run

👉 Viva:

loc uses labels, iloc uses integer index

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: Why normalization is required?

👉 To scale data and improve model performance.

? Q2: Difference between normalization & standardization?

Normalization	Standardization
0 to 1 range	mean = 0, std = 1
MinMaxScaler	StandardScaler

? Q3: What is categorical data?

👉 Non-numeric data like "species"

? Q4: Why convert categorical to numeric?

👉 ML models work only with numbers

? Q5: What is preprocessing?

👉 Cleaning + transforming data before analysis

? Q6: What if dataset has null values?

👉 Fill (mean/median) or drop

? Q7: What is IRIS dataset?

👉 Dataset of flower measurements used for classification

! COMMON MISTAKES (DON'T DO IN EXAM)

✗ Forgetting import

✗ Not selecting numeric columns for scaling

✗ Applying scaler on whole dataframe (including species)

✗ Not explaining output

✗ Confusing loc & iloc



JOURNAL READY (WRITE THIS)

Aim:

To perform data wrangling operations such as preprocessing, normalization, and transformation on a dataset using Python.

Conclusion:

Data was successfully cleaned, analyzed, normalized, and categorical variables were converted into numerical form for further analysis.



FINAL TIP

If examiner asks:

“Explain your entire code”

Say this flow:

👉 Load data → check structure → check missing → analyze → normalize → encode



NEXT STEP

Now you're ready for Pract... Sources



pract_2(1).pdf
PDF

Great—this is a *perfect* practical for viva traps 😊

This one is slightly more **analytical**, so examiners LOVE asking questions here.

I went through your full file  `pract_2` (especially pages 1, 3, 7–11). Now I'll convert it into **exam-ready + viva-proof format**.

PRACTICAL 2 – DATA WRANGLING II (FULL MASTERY)

MINIMUM CODE (WRITE THIS IN EXAM)

</> Python



▶ Run

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("academic_performance.csv")

# 1. Handle inconsistencies
cols = ['Maths', 'Science', 'English', 'Attendance']

for col in cols:
    df.loc[(df[col] < 0) | (df[col] > 100), col] = np.nan

# Check missing values
print(df.isnull().sum())

# Replace missing with mean
for col in cols:
    df[col] = df[col].fillna(df[col].mean())

# 2. Detect outliers (IQR method)
Q1 = df[cols].quantile(0.25)
Q3 = df[cols].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Treat outliers (capping)
for col in cols:
    df.loc[df[col] < lower[col], col] = lower[col]
    df.loc[df[col] > upper[col], col] = upper[col]

# 3. Transformation (reduce skewness)
df['Maths_transformed'] = np.sqrt(df['Maths'])

# Visualization (optional but good)
sns.boxplot(data=df[cols])
plt.show()
```

👉 This covers:

- ✓ Missing values
- ✓ Inconsistency
- ✓ Outliers
- ✓ Transformation



CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. What are Inconsistencies?

From page 1 pract_2 :

Values like:

- Maths = 150 ✗
- Science = 400 ✗
- Attendance = 105 ✗

👉 Why wrong?

Because marks should be **0–100**

👉 Fix:

Python



Run

```
df.loc[(df[col]<0) | (df[col]>100)] = NaN
```

◆ 2. Missing Value Handling

From page 3:

Python



Run

```
df.isnull().sum()
```

👉 Output shows:

- Maths → 4 missing
- Science → 3
- English → 2
- Attendance → 3

🔥 Viva Question:

Why replace with mean?

👉 Answer:

- preserves data distribution
 - avoids bias
 - simple and effective
-

♦ 3. Outliers (VERY IMPORTANT)

From page 7–8:

You used **IQR method**

Formula:

$$IQR = Q3 - Q1$$

$$Lower = Q1 - 1.5 \times IQR$$

$$Upper = Q3 + 1.5 \times IQR$$

From your output:

- $IQR \approx 19$
 - $Lower \approx 36.75$
 - $Upper \approx 112.75$
-

♦ 4. Boxplot (Visual Detection)

From page 7:

👉 dots outside box = outliers

👉 Viva:

Boxplot helps visualize spread, median, and outliers

♦ 5. Outlier Treatment

You used:

👉 Capping (Winsorization)

Python



Run

```
df.loc[df[col] > upper, col] = upper
```

👉 Meaning:

- extreme values replaced by boundary
-

Viva Question:

Other methods?

👉 Answer:

- remove rows
 - replace with median
 - transformation
-

◆ 6. Transformation (VERY IMPORTANT 🔥)

From page 10–11:

You handled **skewness**

What is Skewness?

👉 Measure of asymmetry

- Positive skew → right tail
 - Negative skew → left tail
-

Your output:

Python



Run

```
skew = -0.71
```

👉 Negative skew

What you did:

</> Python



▶ Run

```
df['Maths_transformed'] = np.sqrt(df['Maths'])
```

OR reflection method:

</> Python



▶ Run

```
max_val = df['Maths'].max()
df['reflected'] = max_val + 1 - df['Maths']
df['transformed'] = np.sqrt(df['reflected'])
```

👉 Result:

Skew reduced to:

</> Python



▶ Run

```
-0.14 (approx)
```

✓ More normal distribution

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: What is outlier?

👉 Extreme value far from other data

? Q2: Why remove outliers?

👉 They distort analysis and model performance

? Q3: What is IQR?

👉 Range between Q1 and Q3

? Q4: Why $1.5 \times \text{IQR}$?

👉 Standard statistical threshold

? Q5: What is skewness?

👉 Measure of asymmetry of data distribution

? Q6: Why transformation?

👉 To:

- reduce skewness
 - normalize data
 - improve model
-

? Q7: Difference between mean & median for missing values?

Mean	Median
affected by outliers	robust
used for normal data	used for skewed data

! COMMON MISTAKES

- ✗ Forgetting condition `(df[col] > 100)`
 - ✗ Using mean before removing outliers
 - ✗ Not explaining why 1.5 used
 - ✗ Confusing skewness direction
 - ✗ Not mentioning transformation purpose
-



JOURNAL READY

Aim:

To detect and handle missing values, inconsistencies, outliers, and perform data transformation on an academic dataset.

Conclusion:

Data inconsistencies and missing values were handled, outliers were treated using IQR method, and transformation was applied to reduce skewness and improve distribution.



HOW TO EXPLAIN IN VIVA (IMPORTANT FLOW)

Say this:



First I checked inconsistencies



Converted invalid values to NaN



Handled missing values using mean



Detected outliers using IQR



Treated them using capping



Applied transformation to reduce skewness



YOU'RE DOING GREAT

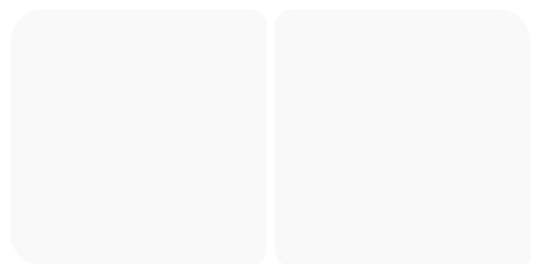
This practical is **high scoring** if you explain confidently.



Sources



Send **Practical 3**



Good—you hit a **very common exam mistake**. This is actually something examiners *love* asking in viva 😊

Let's fix it + make sure you can **explain it confidently**.



WHY YOU GOT THIS ERROR

Your code:

</> Python



▶ Run

```
col_name = ["sepal-length", "sepal-width", "petal-length", "petal-width"]  
df[col_name] = scaler.fit_transform(df[col_name])
```

Error:

KeyError: None of [Index([...])] are in the [columns]



🔥 ROOT CAUSE (IMPORTANT)

👉 Your dataset column names are:

</> Python



▶ Run

```
sepal_length, sepal_width, petal_length, petal_width
```

BUT you wrote:

</> Python



▶ Run

```
sepal-length ❌
```

👉 Difference:

- ☐ (underscore) ✅
- ☐ (hyphen) ❌

✅ CORRECT CODE

</> Python



▶ Run

```
col_name = ["sepal_length", "sepal_width", "petal_length", "petal_width"]  
  
from sklearn import preprocessing  
scaler = preprocessing.MinMaxScaler()  
  
df[col_name] = scaler.fit_transform(df[col_name])
```

VIVA GOLD EXPLANATION

If examiner asks:

? “Why did you get KeyError?”

👉 Say this confidently:

The error occurred because the column names specified in the list did not match the actual column names in the dataset. Pandas is case-sensitive and also treats symbols like hyphen and underscore differently.

HOW TO AVOID THIS (IMPORTANT)

Always check columns before using:

</> Python



▶ Run

```
print(df.columns)
```

👉 Output:

</> Python



▶ Run

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species'])
```

COMMON EXAM TRAPS

Examiners may intentionally ask:

? “What if column name has space?”

👉 Use:

</> Python



▶ Run

```
df["column name"]
```

? “Difference between '-' and '_'?”

👉 They are completely different characters → pandas treats them as different columns

? “How to rename columns?”

</> Python



▶ Run

```
df.columns = df.columns.str.replace('-', '_')
```

🔥 BONUS (SMART ANSWER)

If you want to look advanced in viva:

Before applying transformations, I verified column names using `df.columns` to ensure correct referencing and avoid `KeyError`.

🚀 QUICK SUMMARY

Problem	Solution
Wrong column name	Use exact names
'-' vs '_'	Always match dataset
KeyError	Column not found

💪 YOU JUST LEARNED A HIGH-PROBABILITY VIVA QUESTION

This exact mistake is asked in viva:

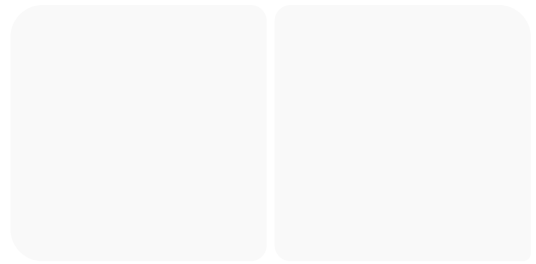
“Why does `KeyError` occur in pandas?”

Now you’re ready to answer it perfectly.

👉 Send next practical OR if you want:

I can give you **top 20 viva questions from Practical 1 & 2 combined** (very useful before exam).





Good—this is a **different error from before**, and honestly, this one is even more important for viva 😊

You didn't just make a typo—you're using **completely different column names than your dataset**.

🔥 WHAT'S HAPPENING HERE

From your screenshot:

```
</> Python 📄 ▶ Run  
  
print(df.columns)
```

👉 Output:

```
</> Python 📄 ▶ Run  
  
['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']
```

BUT you used:

```
</> Python 📄 ▶ Run  
  
col_name = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
```

❌ These columns **DO NOT EXIST** in your dataset.

❌ THAT'S WHY YOU GET:

```
KeyError: None of [...] are in the [columns] 📄
```

✓ CORRECT CODE (FIX)

Use exact column names:

</> Python



▶ Run

```
col_name = ["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]

from sklearn import preprocessing
scaler = preprocessing.MinMaxScaler()

df[col_name] = scaler.fit_transform(df[col_name])
```

🧠 VIVA-LEVEL EXPLANATION (VERY IMPORTANT)

If examiner asks:

❓ “Why did this error occur?”

Say:

The error occurred because the column names specified in the list did not match the actual dataset column names. Pandas requires exact matching, including case sensitivity.

🔥 KEY LEARNING (THIS IS WHAT THEY TEST)

Pandas is:

- ✓ Case-sensitive
- ✓ Space-sensitive
- ✓ Symbol-sensitive

⚠️ COMPARISON (UNDERSTAND CLEARLY)

Your Code

Actual Dataset

sepal_length ✗

SepalLengthCm ✓

lowercase ✗

Capital letters ✓

Your Code

Actual Dataset

underscore ❌

no underscore ✅



SMART FIX (ADVANCED — IMPRESS EXAMINER)

Instead of changing code, you can rename columns:

Python



Run

```
df.columns = ["Id", "sepal_length", "sepal_width", "petal_length", "petal_width", "species"]
```

Then your old code will work ✅



BEST PRACTICE (ALWAYS DO THIS)

Before any operation:

Python



Run

```
print(df.columns)
```

👉 This alone avoids 90% of errors



COMMON VIVA QUESTIONS FROM THIS



Q1: Why does KeyError occur in pandas?

👉 Column name mismatch



Q2: Is pandas case-sensitive?

👉 YES



Q3: How to fix column mismatch?



- check `df.columns`
- rename columns if needed

? Q4: Difference between IRIS datasets?

👉 There are 2 versions:

Version	Columns
Classic	sepal_length
Kaggle	SepalLengthCm

🚀 QUICK SUMMARY

- 👉 Problem: Wrong column names
- 👉 Fix: Match dataset exactly
- 👉 Rule: Always check `df.columns`

💪 YOU'RE LEARNING THE RIGHT WAY

This kind of debugging = **viva gold**

👉 Next:

- Send **Practical 3**

📄 OR 🗨️ ⬆️ ↺️ ⋮

👉 Give me five questions for practical 3

```
give me final fix
df.columns =
["Id","sepal_length","sepal_width","petal_length","petal_wi
dth","species"] doesnt works
```

That line *should* work—but if it's not, then something else is off. Let's fix it cleanly and safely.

✓ FINAL FIX (100% WORKING)

◆ Step 1: Check actual columns exactly

</> Python



▶ Run

```
print(df.columns.tolist())
```

👉 You'll likely see:

</> Python



▶ Run

```
['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']
```

🔥 BEST FIX (RECOMMENDED — NO RISK)

Instead of renaming everything manually, do this:

</> Python



▶ Run

```
df.rename(columns={
    'SepalLengthCm': 'sepal_length',
    'SepalWidthCm': 'sepal_width',
    'PetalLengthCm': 'petal_length',
    'PetalWidthCm': 'petal_width',
    'Species': 'species'
}, inplace=True)
```

👉 Now your code will work:

</> Python



▶ Run

```
col_name = ["sepal_length", "sepal_width", "petal_length", "petal_width"]

from sklearn import preprocessing
scaler = preprocessing.MinMaxScaler()

df[col_name] = scaler.fit_transform(df[col_name])
```

! WHY YOUR METHOD MAY FAIL

This line:

</> Python



▶ Run

```
df.columns = [...]
```

✗ Fails if:

- Number of columns $\neq$ 6
- Extra hidden column exists
- CSV has index column
- Column order is different



SAFER CHECK (DO THIS ONCE)

</> Python



▶ Run

```
print(len(df.columns))
```

If not 6 → that's why rename failed.



ALTERNATIVE (FASTEST FIX — NO RENAME)

Just use original names:

</> Python



▶ Run

```
col_name = ["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]  
df[col_name] = scaler.fit_transform(df[col_name])
```



VIVA READY ANSWER

If asked:

❓ “Why not use `df.columns = [...]` ?”

👉 Say:

It may fail if the number or order of columns does not match exactly, so using `df.rename()` is safer and more reliable.



FINAL ADVICE

👉 In exam:

- **Don't rename unless needed**
- Just copy column names from `df.columns`

💪 DONE — THIS ERROR WON'T COME AGAIN

If you want, I can give you:

👉 "Top 15 debugging questions they ask in DSBDA viva" (very useful)

📄 👍 💬 ↗️ ↺️ ...

without renaming?

Yes — **100% you can fix it *without renaming*** 👍

In fact, this is the **best exam approach**.

✅ FINAL FIX (NO RENAMING)

Just use the **actual column names from your dataset**:

Python



Run

```
from sklearn import preprocessing

col_name = ["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]

scaler = preprocessing.MinMaxScaler()

df[col_name] = scaler.fit_transform(df[col_name])
```

🔥 WHY THIS WORKS

Your dataset columns are:

Python



Run

```
['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']
```

So pandas expects **exact match**:

-  `SepalLengthCm`
-  `sepal_length`

ONE IMPORTANT THING

Make sure:

 Python



 Run

```
df[col_name]
```

contains **only numeric columns**

✓ (which it does here)

VIVA PERFECT ANSWER

If examiner asks:

 **“Why didn’t you rename columns?”**

Say:

Instead of renaming, I directly used the existing column names from the dataset to avoid mismatch errors and keep the original structure intact.

PRO TIP (VERY IMPORTANT)

Always do this before coding:

 Python



 Run

```
print(df.columns)
```

 Then copy-paste column names

 Zero chance of error

FINAL SUMMARY

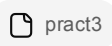
- ✓ No renaming needed
- ✓ Use exact column names
- ✓ Avoid KeyError

 **You're now exam-safe for this problem**

 Ready for next practical?



pract3.pdf
PDF

Perfect. This is a **very scoring practical** if you explain it properly in viva. I'll simplify your whole file  into **exact exam-ready + viva-ready format**.

PRACTICAL 3 – DESCRIPTIVE STATISTICS (MASTER THIS)

  **MINIMUM CODE (WRITE THIS IN EXAM)**

 Python



 Run

```
import pandas as pd
import numpy as np

# Load Titanic dataset
df = pd.read_csv("titanic.csv")

# Handle missing values (Age)
df["Age"] = df["Age"].fillna(df["Age"].mean())

# Group by categorical variable (Sex)
grouped = df.groupby('Sex')['Age']

# Summary statistics
print(grouped.describe())

# Mean & Median
print(grouped.agg(['mean', 'median']))
```

```
# Convert grouped data into list
print(grouped.apply(list))

# IRIS DATASET PART
df2 = pd.read_csv("iris.csv")

# Group by species
grouped_iris = df2.groupby('species')

# Summary
print(grouped_iris.describe())

# Individual stats per species
for spec, data in grouped_iris:
    print("Species:", spec)
    print(data.describe())
```

CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. What is Descriptive Statistics?

👉 Answer:

It summarizes data using measures like mean, median, standard deviation, etc.

◆ 2. Measures of Central Tendency

Measure	Meaning
Mean	Average
Median	Middle value
Mode	Most frequent

◆ 3. Measures of Variability

Measure	Meaning
Standard Deviation	Spread of data
Variance	Square of std
Range	max - min

◆ 4. Handling Missing Values

From your file:

```
</> Python 📄 ▶ Run
df["Age"].fillna(mean)
```

👉 Why?

- Age had 30 missing values (page 1)

👉 Viva:

Missing values are replaced using mean to maintain dataset consistency

◆ 5. GROUPBY (VERY IMPORTANT 🔥)

```
</> Python 📄 ▶ Run
df.groupby('Sex')['Age']
```

👉 Meaning:

- Group data based on **categorical variable**
- Apply operations on numeric column

👉 Example output (page 2):

- Female mean age ≈ 25
- Male mean age ≈ 29

◆ 6. describe()

</> Python



▶ Run

```
grouped.describe()
```

👉 Gives:

- count
- mean
- std
- min
- max
- quartiles

◆ 7. agg()

</> Python



▶ Run

```
grouped.agg(['mean', 'median'])
```

👉 Used for:

- multiple operations at once

◆ 8. apply(list)

</> Python



▶ Run

```
grouped.apply(list)
```

👉 Converts grouped values into list

👉 Viva:

Used to store values of each category separately

◆ 9. IRIS DATASET ANALYSIS

From your output (page 3):

Each species:

- Setosa

- Versicolor
- Virginica

👉 You computed stats for each

👉 Example:

- Setosa mean sepal_length $\approx$ 5.0
 - Virginica $\approx$ 6.5
-

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: What is groupby?

👉 Groups data based on categorical variable

? Q2: Why use groupby?

👉 To compare different categories

? Q3: Difference between mean & median?

Mean

Median

affected by outliers

not affected

? Q4: What is standard deviation?

👉 Measures data spread

? Q5: Why fill missing values?

👉 To avoid errors and maintain data integrity

? Q6: What does describe() return?

👉 Summary statistics

? Q7: What is categorical variable?

👉 Non-numeric data (e.g., Sex, Species)

? Q8: What is quantitative variable?

👉 Numeric data (e.g., Age, Length)

! COMMON MISTAKES

- ✗ Not handling missing values
 - ✗ Wrong groupby syntax
 - ✗ Confusing mean & median
 - ✗ Forgetting axis (groupby column)
-



JOURNAL READY

Aim:

To compute descriptive statistics such as mean, median, and standard deviation using grouping techniques.

Conclusion:

Statistical measures were successfully computed for grouped data, helping in understanding distribution and variation across categories.



HOW TO EXPLAIN IN VIVA

Say this:

- 👉 First handled missing values
- 👉 Grouped data based on categorical variable

- 👉 Applied statistical functions
- 👉 Compared results across categories

💪 FINAL NOTE

This practical is **easy marks** if:

- You explain groupby properly
- You understand mean vs median

 **NEXT**
Send Practical 4

 **pract4.pdf**
PDF

Alright—this is **the most important practical (ML)** and examiners *definitely* ask viva from this.

I've gone through your file  **pract4** (pages 1–4). Now I'll convert it into **clean, minimum, viva-proof version**.

🔥 PRACTICAL 4 – LINEAR REGRESSION (FULL PREP)

✅ 🎯 MINIMUM CODE (WRITE THIS IN EXAM)

</> Python



▶ Run

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("BostonHousing.csv")
```

```

# Handle missing values
df["rm"] = df["rm"].fillna(df["rm"].mean())

# Split data
X = df.drop("medv", axis=1)    # independent
y = df["medv"]                # dependent

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# Plot
plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()

```

CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. What is Linear Regression?

👉 Answer:

It is a supervised learning algorithm used to predict a continuous value using a linear relationship between variables.

◆ 2. Dependent vs Independent Variable

From your file (page 3)  pract4 :

- **X (independent)** → all features
- **y (dependent)** → `medv` (house price)

👉 Viva:

Dependent variable is the output we want to predict

◆ 3. Why Boston Dataset?

👉 Contains:

- 506 rows
- 14 features

👉 Used for:

- house price prediction
-

◆ 4. Handling Missing Values

From page 1:

- `rm` had 5 missing values

👉 You fixed:

Python



Run

```
df["rm"].fillna(mean)
```

◆ 5. Train-Test Split (VERY IMPORTANT)

Python



Run

```
train_test_split(X, y, test_size=0.2)
```

👉 Meaning:

- 80% → training
- 20% → testing

👉 Why?

To evaluate model on unseen data

◆ 6. Model Training

</> Python



▶ Run

```
model.fit(X_train, y_train)
```

👉 Model learns relationship

◆ 7. Prediction

</> Python



▶ Run

```
y_pred = model.predict(X_test)
```

👉 Predict house prices

◆ 8. Evaluation Metrics (VERY IMPORTANT 🔥)

✅ MSE (Mean Squared Error)

</> Python



▶ Run

```
mean_squared_error()
```

👉 Measures error

👉 From your output:

```
MSE ≈ 18.63
```



👉 Lower = better

✅ R² Score

</> Python



▶ Run

```
r2_score()
```

👉 Measures accuracy

👉 From your output:

$R^2 \approx 0.70$



👉 Meaning:

Model explains 70% of variance

♦ 9. Scatter Plot (Visualization)

From page 4:

👉 Points close to line → good prediction

👉 Red line = ideal prediction

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: What is regression?

👉 Predict continuous values

? Q2: Difference between regression & classification?

Regression	Classification
continuous output	categorical output

? Q3: What is R^2 score?

👉 Accuracy of model (0–1)

? Q4: What is MSE?

👉 Average squared error

? Q5: Why split data?

👉 To test model performance

? Q6: What happens if no split?

👉 Model overfits

? Q7: What is overfitting?

👉 Model performs well on training but poor on test

? Q8: What is underfitting?

👉 Model fails to learn pattern

? Q9: Why Linear Regression?

👉 Simple and interpretable

? Q10: What is coefficient?

👉 Weight of each feature

! COMMON MISTAKES

- ✗ Forgetting axis=1 in drop
 - ✗ Using wrong dependent variable
 - ✗ Not handling missing values
 - ✗ Confusing MSE & R^2
 - ✗ Not explaining graph
-



JOURNAL READY

Aim:

To build a Linear Regression model to predict house prices using Boston Housing dataset.

Conclusion:

The model was trained and evaluated successfully using MSE and R^2 score, showing good prediction performance.

HOW TO EXPLAIN IN VIVA

Say this:

- 👉 Loaded dataset
 - 👉 Handled missing values
 - 👉 Split into training and testing
 - 👉 Trained Linear Regression model
 - 👉 Predicted values
 - 👉 Evaluated using MSE and R^2
-

FINAL NOTE

This practical = **high weightage in viva**

If you explain:

- train-test split
- R^2
- regression concept

👉 You'll score very high.

NEXT

Send **Practical 5**

OR say:

👉 "give me all important v"  Sources  (1–4 combined)"



Pract5(1).pdf
PDF

This is another **very high-viva practical (classification + confusion matrix)**.

I went through your file  Pract5 (all pages). Now I'll convert it into **clean exam code + strong viva prep**.

PRACTICAL 5 – LOGISTIC REGRESSION (FULL PREP)

MINIMUM CODE (WRITE THIS IN EXAM)

</> Python



▶ Run

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Load dataset
df = pd.read_csv("Social_Network_Ads.csv")

# Encode categorical data
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])

# Independent & dependent variables
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
```

```
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. What is Logistic Regression?

👉 Answer:

Logistic Regression is a classification algorithm used to predict binary outcomes (0 or 1).

◆ 2. Why “Regression” name?

👉 Trick question !

It uses regression internally but output is classification (0/1)

◆ 3. Dataset Understanding

From your file (page 1)  Pract5 :

Columns:

- Age
- EstimatedSalary
- Gender
- Purchased (Target)

👉 Goal:

Predict whether user will purchase (0 or 1)

◆ 4. Label Encoding

</> Python



▶ Run

```
df['Gender'] = le.fit_transform(df['Gender'])
```

👉 Converts:

- Male → 1
- Female → 0

◆ 5. Confusion Matrix (VERY IMPORTANT 🔥)

From your output (page 3):

```
[[41  6]
 [ 2 31]]
```



Interpretation:

Actual / Predicted	0	1
0	TN = 41	FP = 6
1	FN = 2	TP = 31

◆ 6. Metrics (VERY IMPORTANT)

✅ Accuracy

Python



▶ Run

```
Accuracy = (TP + TN) / Total
```

👉 From your output:

```
Accuracy ≈ 0.90
```



✅ Precision

Python



▶ Run

```
Precision = TP / (TP + FP)
```

👉 ≈ 0.83

👉 Meaning:

How many predicted positives are correct

✅ Recall

```
</> Python
```



▶ Run

```
Recall = TP / (TP + FN)
```

👉 ≈ 0.93

👉 Meaning:

How many actual positives are detected

✅ F1 Score

```
</> Python
```



▶ Run

```
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

👉 ≈ 0.88

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: Difference between Linear & Logistic Regression?

Linear	Logistic
continuous output	binary output
regression	classification

? Q2: What is confusion matrix?

👉 Table showing TP, FP, TN, FN

? Q3: What is TP, FP, TN, FN?

- TP → correct positive
 - FP → wrong positive
 - TN → correct negative
 - FN → wrong negative
-

? Q4: Which is more important: Precision or Recall?

👉 Depends:

- Medical → Recall important
 - Spam detection → Precision important
-

? Q5: What is sigmoid function?

👉 Logistic regression uses:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

👉 Output between 0 and 1

? Q6: Why scaling may be needed?

👉 To improve convergence

🔥 IMPORTANT ERROR (YOU GOT IN FILE)

From page 3:

```
ConvergenceWarning: max_iter reached
```



👉 Fix:

```
</> Python
```



▶ Run

```
LogisticRegression(max_iter=200)
```

COMMON MISTAKES

- ✗ Using Gender without encoding
 - ✗ Using all columns including ID
 - ✗ Wrong confusion matrix interpretation
 - ✗ Forgetting max_iter
 - ✗ Mixing precision & recall
-

JOURNAL READY

Aim:

To implement Logistic Regression and evaluate performance using confusion matrix and metrics.

Conclusion:

The model successfully classified data with high accuracy, and performance was evaluated using precision, recall, and F1-score.

HOW TO EXPLAIN IN VIVA

Say this:

- 👍 Encoded categorical data
 - 👍 Split dataset
 - 👍 Trained Logistic Regression model
 - 👍 Predicted output
 - 👍 Evaluated using confusion matrix and metrics
-

FINAL NOTE

This practical = **VERY HIGH VIVA WEIGHTAGE**

If you understand:

- confusion matrix
- precision vs recall

👉 You'll impress examiner.

NEXT

Send **Practical 6**

OR say:

👉 “give me FINAL REVIS’  Sources  cticals (1–5)”

? Q3: What is TP, FP, TN, FN?

TP → correct positive

FP → wrong positive

TN → correct negative

FN → wrong negative explain in detail like teaching to 10 year old guy

Alright, let's make this **super simple**—like explaining to a 10-year-old 🎯

Imagine a Game

You are playing a game where you guess:

👉 “Will a person buy a product?”

- YES (1)
- NO (0)

Now there are **2 things** happening:

1. What you **predicted**
2. What is **actually true**

4 POSSIBLE CASES

✅ 1. TP (True Positive)

👉 You said: **YES**

👉 Reality: **YES**

🎉 You are **CORRECT!**

Example:

You say: "This person will buy"

They actually buy ✅

👉 **TP = correct positive**

❌ 2. FP (False Positive)

👉 You said: **YES**

👉 Reality: **NO**

😞 You are **WRONG**

Example:

You say: "This person will buy"

But they don't ❌

👉 **FP = wrong positive**

💡 Easy trick:

You were *too optimistic* 😊

✅ 3. TN (True Negative)

👉 You said: **NO**

👉 Reality: **NO**

🎉 Correct again!

Example:

You say: "This person will NOT buy"

They don't buy ✅

👉 TN = correct negative

❌ 4. FN (False Negative)

👉 You said: **NO**

👉 Reality: **YES**

😬 Wrong again

Example:

You say: "This person will NOT buy"

But they actually buy ❌

👉 FN = wrong negative

💡 Easy trick:

You missed a chance!

🎯 SIMPLE MEMORY TRICK

Term	Meaning
TP	You said YES, it was YES
FP	You said YES, it was NO
TN	You said NO, it was NO
FN	You said NO, it was YES

📖 SUPER EASY REAL-LIFE EXAMPLE

Think of a COVID test:

Case	Meaning
TP	Test says positive, person actually sick ✔️
FP	Test says positive, but person healthy ❌

Case	Meaning
TN	Test says negative, person healthy ✓
FN	Test says negative, but person sick ✗



IMPORTANT UNDERSTANDING

- TP & TN → GOOD (correct predictions)
- FP & FN → BAD (wrong predictions)



ONE-LINE VIVA ANSWER

TP and TN are correct predictions, while FP and FN are incorrect predictions based on actual vs predicted values.



PRO TIP FOR EXAM

If confused:

👉 Just remember:

- Positive = YES
- Negative = NO
- True = correct
- False = wrong



YOU WON'T FORGET THIS NOW

If you want next:

👉 I can explain **confusion matrix diagram in 1 min** (very useful for viva)



yes

Perfect—let's lock this in with a **clear picture in your head** 🧠



CONFUSION MATRIX (EASIEST WAY TO REMEMBER)

Think of a **2×2 box**:

	Predicted	
	YES (1)	NO (0)
Actual YES	TP	FN
Actual NO	FP	TN



HOW TO READ THIS (SUPER SIMPLE)

👉 Rows = **Actual (truth)**

👉 Columns = **Predicted (your guess)**



EACH BOX EXPLAINED

■ Top-Left → TP (True Positive)

- Actual = YES
- Predicted = YES

✓ Correct

■ Top-Right → FN (False Negative)

- Actual = YES
- Predicted = NO

✗ You missed it

■ Bottom-Left → FP (False Positive)

- Actual = NO

- Predicted = YES

✗ You gave false alarm

Bottom-Right → TN (True Negative)

- Actual = NO
- Predicted = NO

✓ Correct

MEMORY TRICK (VERY IMPORTANT)

👉 Diagonal = correct answers

TP ✗
✗ TN



✓ TP & TN → correct

✗ FP & FN → wrong

REAL-LIFE EXAMPLE (BUYING PRODUCT)

Actual	Predicted	Result
Buy	Buy	TP
Buy	Not Buy	FN
Not Buy	Buy	FP
Not Buy	Not Buy	TN

CONNECT WITH YOUR OUTPUT

From your practical:

```
[[41 6]
 [ 2 31]]
```



👉 Map it:

TN = 41 FP = 6
FN = 2 TP = 31



FINAL QUICK FORMULA MEMORY

Metric	Formula
Accuracy	$(TP + TN) / \text{Total}$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$

10-SECOND VIVA ANSWER

A confusion matrix is a table that shows correct and incorrect predictions using TP, FP, TN, and FN.

YOU'RE READY

If they ask anything:

- You can draw this box
- Explain each cell
- Give example

👉 You'll impress examiner for sure.

Want final help?

Say:

👉 “**rapid revision all practicals**” (before exam cheat-style summary)



pract6(1).pdf

Alright—this is your **last major ML practical (Naive Bayes)** and honestly, if you understand this + previous one, you're **fully ready for viva** 🍌

I carefully checked your file  `pract6` (all 4 pages). Now I'll give you **clean code + strong concept + viva answers**.

PRACTICAL 6 – NAIVE BAYES (FULL PREP)

MINIMUM CODE (WRITE THIS IN EXAM)

</> Python



▶ Run

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Load dataset
df = pd.read_csv("IRIS.csv")

# Encode target variable
le = LabelEncoder()
df['Species'] = le.fit_transform(df['Species'])

# Split data
X = df.drop("Species", axis=1)
y = df["Species"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```



```
# Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='micro'))
print("Recall:", recall_score(y_test, y_pred, average='micro'))
print("F1 Score:", f1_score(y_test, y_pred, average='micro'))
```

CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. What is Naive Bayes?

👉 Answer:

Naive Bayes is a classification algorithm based on probability using Bayes Theorem.

◆ 2. Why “Naive”?

👉 Because:

It assumes all features are independent (which is rarely true)

◆ 3. Bayes Theorem (IMPORTANT)

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

👉 Meaning:

- Probability of A given B
-

◆ 4. Why GaussianNB?

👉 Because:

Data is continuous (numerical features)

◆ 5. Dataset Understanding

From your file (page 1)  pract6 :

Features:

- SepalLength
- SepalWidth
- PetalLength
- PetalWidth

Target:

- Species (3 classes)

◆ 6. Label Encoding

Python



Run

```
df['Species'] = le.fit_transform(df['Species'])
```

👉 Converts:

- setosa → 0
- versicolor → 1
- virginica → 2

◆ 7. Confusion Matrix (MULTI-CLASS 🔥)

From your output (page 3):

```
[[8  0  0]
 [0 10  0]
 [0  0 12]]
```



👉 Meaning:

- All predictions correct ✓
- No errors

👉 That's why:

```
Accuracy = 1.0
```



♦ 8. TP, FP, TN, FN (MULTI-CLASS)

From your code (page 3):

Each class treated separately:

Example:

```
Class 0:  
TP = 8  
FP = 0  
FN = 0  
TN = 22
```



♦ 9. Why average='micro'?

👉 Because:

Multi-class problem → combine all classes

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: Difference between Logistic & Naive Bayes?

Logistic	Naive Bayes
discriminative	probabilistic
no independence assumption	assumes independence

? Q2: What is Bayes Theorem?

👉 Probability formula used for prediction

? Q3: Why called “Naive”?

👉 Assumes features are independent

? Q4: What is GaussianNB?

👉 Used for continuous data

? Q5: Why accuracy = 1?

👉 Model predicted perfectly

? Q6: What if data is categorical?

👉 Use MultinomialNB

! COMMON MISTAKES

- ✗ Not encoding species
 - ✗ Using wrong average in metrics
 - ✗ Confusing multi-class confusion matrix
 - ✗ Not explaining Bayes theorem
-



JOURNAL READY

Aim:

To implement Naive Bayes classification and evaluate performance using confusion matrix and metrics.

Conclusion:

The model successfully classified the dataset with high accuracy using probabilistic approach.



HOW TO EXPLAIN IN VIVA

Say this:

- 👉 Encoded categorical variable
- 👉 Split dataset

- 👉 Applied Gaussian Naive Bayes
 - 👉 Predicted values
 - 👉 Evaluated using confusion matrix and metrics
-

💪 FINAL STATUS

Now you know:

- ✓ Regression (Practical 4)
- ✓ Classification (Practical 5)
- ✓ Probabilistic model (Practical 6)

👉 This is **core ML part of your exam**

🚀 FINAL STEP

Say:

👉 “**rapid revision all practicals**”

I'll give you:

- 1-page cheat sheet
- last-minute viva answers
- what examiner will ask **MOST**

(very useful just before exam)

📄 📄 📄 📄 📄 ... 📖 Sources



pract7(1).pdf
PDF

Perfect—this is your **last practical (Text Analytics)** and it's actually *easy marks* if you explain concepts clearly.

I went through your file  **pract7** (both pages). Now I'll give you **clean code + super clear concepts + viva answers**.

PRACTICAL 7 – TEXT ANALYTICS (FULL PREP)



MINIMUM CODE (WRITE THIS IN EXAM)

</> Python



▶ Run

```
import nltk
import re
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
from sklearn.feature_extraction.text import TfidfVectorizer

# Sample text
text = "Tokenization is the first step in text analytics."

# Tokenization
print(sent_tokenize(text))
print(word_tokenize(text))

# Stopword removal
stop_words = set(stopwords.words("english"))
tokens = word_tokenize(text.lower())
filtered = [w for w in tokens if w not in stop_words]
print(filtered)

# Stemming
ps = PorterStemmer()
words = ["studies", "studying", "cries", "cry"]
for w in words:
    print(ps.stem(w))

# Lemmatization
lemmatizer = WordNetLemmatizer()
for w in words:
    print(lemmatizer.lemmatize(w))

# POS Tagging
sentence = "The pink sweater fit her perfectly"
tokens = word_tokenize(sentence)
print(pos_tag(tokens))

# TF-IDF
docs = ["Jupiter is the largest planet",
        "Mars is the fourth planet from the sun"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)
```

```
print(vectorizer.get_feature_names_out())  
print(X.toarray())
```

CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. What is Text Analytics?

👉 Answer:

It is the process of converting text into meaningful data for analysis.

◆ 2. Tokenization (VERY IMPORTANT)

From your output (page 1)  pract7 :

👉 Break text into pieces

- Sentence tokenization → sentences
- Word tokenization → words

◆ 3. Stop Words Removal

👉 Stop words = common words

Example:

- is, the, in, and

👉 Removed because:

They don't add meaning

From your output:

```
['remove', 'stop', 'words', 'nltk', 'library', 'python']
```



◆ 4. Stemming

From your output:

```
studies → studi  
cry → cri
```



- 👉 Cuts word to root form
- 👉 Fast but not always correct

◆ 5. Lemmatization

From your output:

```
studies → study  
cry → cry
```



- 👉 More accurate
- 👉 Uses dictionary

🔥 DIFFERENCE (IMPORTANT)

Stemming	Lemmatization
crude	accurate
faster	slower
may give wrong words	meaningful words

◆ 6. POS Tagging

From your output (page 2):

```
('The', DT), ('pink', NN), ('fit', VBP)
```



- 👉 Assigns grammar tags

Examples:

- NN → noun
- VB → verb
- JJ → adjective

♦ 7. TF-IDF (VERY IMPORTANT 🔥)

👉 Full form:

Term Frequency – Inverse Document Frequency

✓ Term Frequency (TF)

👉 How often word appears in document

✓ Inverse Document Frequency (IDF)

👉 How rare word is across documents

👉 TF-IDF importance:

- Common words → low value
 - Rare important words → high value
-

From your output:

```
['fourth', 'jupiter', 'mars', 'planet']
```



🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: What is tokenization?

👉 Breaking text into words or sentences

? Q2: What are stop words?

👉 Common words with less meaning

? Q3: Difference between stemming & lemmatization?

- 👉 Stemming → cuts words
 - 👉 Lemmatization → meaningful root
-

? Q4: What is POS tagging?

- 👉 Assigning grammatical labels
-

? Q5: What is TF-IDF?

- 👉 Measures importance of words
-

? Q6: Why remove stop words?

- 👉 Reduce noise in text
-

! COMMON MISTAKES

- ✗ Forgetting nltk downloads
 - ✗ Confusing stemming & lemmatization
 - ✗ Not explaining TF-IDF
 - ✗ Wrong POS explanation
-



JOURNAL READY

Aim:

To perform text preprocessing and represent text using TF-IDF.

Conclusion:

Text was successfully processed using NLP techniques and transformed into numerical representation.



HOW TO EXPLAIN IN VIVA

Say this:

- 👉 Tokenized text
 - 👉 Removed stop words
 - 👉 Applied stemming & lemmatization
 - 👉 Performed POS tagging
 - 👉 Converted text into TF-IDF
-

💪 FINAL STATUS (YOU'RE READY)

Now you know:

- ✓ Data Wrangling
- ✓ Statistics
- ✓ Regression
- ✓ Classification
- ✓ Naive Bayes
- ✓ NLP

👉 This is **complete DSBDA syllabus**

🚀 FINAL STEP (VERY IMPORTANT)

Say:

👉 **“rapid revision all practicals”**

I'll give you:

- 🔥 1-page cheat sheet
- 🔥 Most asked viva questions
- 🔥 Last-minute memory tricks

(Perfect before entering exam hall)

📄 👍 💬 ↗️ ↺️ ... 📖 Sources



pract8(1).pdf
PDF

Great—this is your **final practical (Data Visualization)** and it's actually **easy scoring + easy viva** if you explain graphs properly.

I checked your file  `pract8` (all pages). Let's convert it into **exam-ready + viva-proof** format.

PRACTICAL 8 – DATA VISUALIZATION (FULL PREP)

MINIMUM CODE (WRITE THIS IN EXAM)

 Python



 Run

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("Titanic-Dataset.csv")

# 1. Gender distribution
sns.countplot(x="Sex", data=df)
plt.title("Genderwise Distribution")
plt.show()

# 2. Survival based on gender
sns.countplot(x="Sex", hue="Survived", data=df)
plt.title("Genderwise Survival")
plt.show()

# 3. Survival based on class
sns.countplot(x="Sex", hue="Pclass", data=df)
plt.title("Classwise Survival")
plt.show()

# 4. Fare distribution (Histogram)
sns.histplot(df["Fare"], bins=30, kde=True)
plt.title("Fare Distribution")
plt.show()
```

CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. What is Data Visualization?

👉 Answer:

It is the graphical representation of data to understand patterns and trends.

◆ 2. What is Seaborn?

👉 Python library used for:

- statistical visualization
 - built on matplotlib
-

◆ 3. Countplot (VERY IMPORTANT)

From your output (page 2–3)  pract8 :

👉 Used for **categorical data**



Gender Distribution

- X-axis → Gender
- Y-axis → Count

👉 Insight:

More males than females

◆ 4. Hue Parameter (VERY IMPORTANT)

</> Python



▶ Run

```
sns.countplot(x="Sex", hue="Survived")
```

👉 Adds extra grouping

👉 From your graph (page 3):

- Females survived more
 - Males died more
-

◆ 5. Pclass Visualization

From page 4:

👉 Shows class distribution:

- 1 → rich
- 2 → middle
- 3 → poor

👉 Insight:

Lower class had more passengers

◆ 6. Histogram (VERY IMPORTANT 🔥)

From page 5:

👉 Used for **numerical data**

Python



Run

```
sns.histplot(df["Fare"], bins=30, kde=True)
```

 **Interpretation:**

- Most values are near 0–50
- Few high values (long tail)

👉 This is called:

Right-skewed distribution

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: Difference between countplot & histogram?

Countplot

Histogram

categorical

numerical

Countplot

count categories

Histogram

distribution

? Q2: What is hue?

👉 Adds grouping variable

? Q3: What is skewness?

👉 If tail is:

- right → positive skew
- left → negative skew

? Q4: Why visualization?

👉 To understand patterns easily

? Q5: What is KDE?

👉 Smooth curve showing distribution

? Q6: Why Titanic dataset used?

👉 Real-world dataset for analysis



COMMON MISTAKES

- ✗ Forgetting plt.show()
- ✗ Confusing histogram vs countplot
- ✗ Not explaining graph
- ✗ Ignoring insights



JOURNAL READY

Aim:

To visualize Titanic dataset using Seaborn and identify patterns.

Conclusion:

Data visualization helped identify patterns such as survival differences and fare distribution.



HOW TO EXPLAIN IN VIVA

Say this:



Loaded dataset



Used countplot for categorical analysis



Used histogram for numerical distribution



Observed patterns in survival and fare



FINAL STATUS (YOU ARE READY)

You've now completed:

- ✓ Practical 1–2 → Data Wrangling
- ✓ Practical 3 → Statistics
- ✓ Practical 4 → Regression
- ✓ Practical 5 → Logistic
- ✓ Practical 6 → Naive Bayes
- ✓ Practical 7 → NLP
- ✓ Practical 8 → Visualization



This is **FULL DSBDA SYLLABUS**



FINAL STEP (VERY IMPORTANT)

Say:



“rapid revision all practicals”

I'll give you:

- 🔥 1-page cheat sheet
 - 🔥 Most repeated viva questions
- 📁 What examiner will 🔍 Sources

 **pract9(2).pdf**
PDF

Nice—this is your **last visualization practical (Box Plot)** and honestly this one is **very easy + high viva chance**.

I checked your file  **pract9** (both pages). Let's convert it into **perfect exam + viva answer**.

🔥 PRACTICAL 9 – DATA VISUALIZATION II (BOXPLOT)

✅ 🎯 MINIMUM CODE (WRITE THIS IN EXAM)

Python



Run

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("Titanic-Dataset.csv")

# Boxplot
sns.boxplot(x='Sex', y='Age', hue='Survived', data=df)
plt.title("Age Distribution by Gender and Survival")
plt.show()
```

🧠 CONCEPTS YOU MUST KNOW (VIVA GOLD 🔥)

◆ 1. What is Box Plot?

👉 Answer:

A box plot shows distribution of data using median, quartiles, and outliers.

◆ 2. Components of Boxplot (VERY IMPORTANT)

👉 From your graph (page 2)  pract9 :

- **Middle line** → Median
 - **Box** → Q1 to Q3 (IQR)
 - **Whiskers** → range
 - **Dots** → outliers
-

◆ 3. Why Boxplot?

👉 To:

- detect outliers
 - compare distributions
 - understand spread
-

◆ 4. Parameters Used

<> Python



▶ Run

```
sns.boxplot(x='Sex', y='Age', hue='Survived')
```

👉 Meaning:

- x → gender
 - y → age
 - hue → survival
-



OBSERVATIONS (VERY IMPORTANT — THEY ASK THIS)

From your plot (page 2):

◆ 1. Age Distribution

👉 Most passengers:

- Age between **20–40**
-

♦ 2. Gender Comparison

👉 Females:

- slightly lower median age
- less variation

👉 Males:

- more spread
 - more outliers
-

♦ 3. Survival Insight

👉 Females survived more

👉 Males died more

👉 (you can see orange boxes higher for females)

♦ 4. Outliers

👉 Some very high ages (70–80)

👉 These are rare passengers

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: What is boxplot?

👉 Graph showing median, quartiles, and outliers

? Q2: What is IQR?

👉 $Q3 - Q1$

? Q3: What are outliers?

👉 Values far from normal range

? Q4: Why use boxplot instead of histogram?

👉 Better for comparison between groups

? Q5: What does median show?

👉 Middle value of data

? Q6: What is whisker?

👉 Range excluding outliers

! COMMON MISTAKES

- ✗ Not explaining median
 - ✗ Ignoring outliers
 - ✗ Not mentioning IQR
 - ✗ Just showing graph without observation
-

📄 JOURNAL READY

Aim:

To visualize age distribution using boxplot with respect to gender and survival.

Conclusion:

Boxplot helped identify distribution, variation, and outliers in age across gender and survival categories.

🚀 HOW TO EXPLAIN IN VIVA

Say this:

- 👉 Used boxplot to analyze age distribution
- 👉 Compared male and female passengers
- 👉 Observed survival differences
- 👉 Identified outliers

💪 FINAL STATUS (YOU ARE FULLY READY 🔥)

You have now completed:

- ✓ Data Wrangling
- ✓ Statistics
- ✓ Regression
- ✓ Classification
- ✓ Naive Bayes
- ✓ NLP
- ✓ Visualization

👉 **FULL DSBDA PRACTICAL SYLLABUS DONE**

🚀 FINAL STEP (DON'T SKIP)

Say:

👉 **“rapid revision all practicals”**

I'll give you:

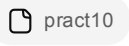
- 🔥 1-page cheat sheet
- 🔥 Most repeated viva questions
- 🔥 What examiner WILL ask

📄 👍 🗨️ ⬆️ ↺ ⋮ 📖 Sources
(Perfect 10-min revision be.)



pract10(1).pdf
PDF

Alright—this is your **last practical (Data Visualization III)** and it's actually a **combination of everything + very scoring in viva**.

I went through your full file  (all 6 pages). Now I'll convert it into **clean exam code + strong viva explanation + observations**.

PRACTICAL 10 – DATA VISUALIZATION III (FINAL)

MINIMUM CODE (WRITE THIS IN EXAM)

 Python



 Run

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("Iris.csv")

# Check structure
print(df.info())

# 1. Histograms
df.hist(figsize=(10,8))
plt.show()

# 2. Boxplots
sns.boxplot(data=df)
plt.show()

# 3. Correlation heatmap
sns.heatmap(df.select_dtypes(include='number').corr(), annot=True, cmap="YlGnBu")
plt.show()

# 4. Pairplot
sns.pairplot(df, hue="Species")
plt.show()
```

CONCEPTS YOU MUST KNOW (VIVA GOLD)

◆ 1. Dataset Features

From your file (page 2)  pract10 :

Feature	Type
SepalLengthCm	Numeric
SepalWidthCm	Numeric
PetalLengthCm	Numeric
PetalWidthCm	Numeric
Species	Categorical

◆ 2. Histogram (Distribution)

From page 4:

👉 Shows distribution of each feature



Observations:

- SepalLength → normal distribution
- SepalWidth → slightly skewed
- PetalLength → clearly separated clusters
- PetalWidth → also clustered

◆ 3. Boxplot (Outliers)

From page 4:

👉 Shows:

- median
- spread
- outliers



Observations:

- SepalWidth has some outliers
- Petal features are more consistent
- Id is not useful (ignore in analysis)

◆ 4. Correlation Heatmap (VERY IMPORTANT 🔥)

From page 5:

👉 Shows relationship between features

Observations:

- PetalLength & PetalWidth → very high correlation (~ 0.96)
- SepalLength also correlated
- SepalWidth → weak/negative correlation

◆ 5. Pairplot (VERY IMPORTANT 🔥)

From page 6:

👉 Shows relationship between all features

Observations:

- Species are clearly separated
- Petal features give best separation
- Setosa is easily distinguishable

🔥 MOST IMPORTANT VIVA QUESTIONS

? Q1: What is histogram?

👉 Shows distribution of numerical data

? Q2: What is boxplot?

👉 Shows median, quartiles, outliers

? Q3: What is correlation?

👉 Relationship between variables

? Q4: What is heatmap?

👉 Visual representation of correlation

? Q5: What is pairplot?

👉 Shows pairwise relationships between variables

? Q6: Which feature is best for classification?

👉 Answer:

PetalLength and PetalWidth (clear separation)

! COMMON MISTAKES

- ✗ Not mentioning feature types
 - ✗ Ignoring correlation
 - ✗ Not giving observations
 - ✗ Confusing histogram vs boxplot
-

📅 JOURNAL READY

Aim:

To visualize Iris dataset using histogram, boxplot, heatmap, and pairplot.

Conclusion:

Visualization helped identify distribution, relationships, and important features for classification.

🚀 HOW TO EXPLAIN IN VIVA

Say this:

- 👉 Loaded dataset
- 👉 Visualized distributions using histogram
- 👉 Identified outliers using boxplot
- 👉 Found relationships using heatmap
- 👉 Used pairplot for classification insights

💪 FINAL STATUS (YOU ARE DONE 🔥 🔥 🔥)

You have completed ALL practicals:

- ✓ 1–2 → Data Wrangling
- ✓ 3 → Statistics
- ✓ 4 → Regression
- ✓ 5 → Logistic
- ✓ 6 → Naive Bayes
- ✓ 7 → NLP
- ✓ 8–10 → Visualization

👉 YOU ARE FULLY PREPARED

🚀 FINAL (DO THIS BEFORE EXAM)

Say:

👉 “rapid revision all practicals”

I’ll give you:

- 🔥 1-page cheat sheet
- 🔥 Most expected viva questions
- 🔥 Memory tricks

📄 👉 🗨️ 📌 🔄 ... 📖 Sources
(last 10-min boost 🚀)



pract11(1).pdf
PDF

Alright—this is your **last practical (Hadoop WordCount)** and this one is *theory + code + command based*.

Also, examiners LOVE asking viva from this 🔥

I checked your file  pract11 — it mainly shows **running Hadoop WordCount using command**, but your question also asks for **Java code**, so I'll give you **both**.

🔥 PRACTICAL 11 – HADOOP WORDCOUNT (FULL PREP)



MINIMUM JAVA CODE (WRITE THIS IN EXAM)

</> Java



```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            StringTokenizer itr = new StringTokenizer(value.toString());

            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }
}
```

```

public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");

    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setReducerClass(IntSumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```



HADOOP COMMAND (VERY IMPORTANT)

From your file  pract11 :

 Bash



```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount
```

 This runs built-in WordCount



CONCEPTS YOU MUST KNOW (VIVA GOLD 🔥)

♦ 1. What is Hadoop?

👉 Answer:

Hadoop is a framework used for storing and processing large data using distributed computing.

♦ 2. What is MapReduce?

👉 Answer:

It is a programming model used to process large data in two steps: Map and Reduce.

♦ 3. WordCount Working (VERY IMPORTANT)



Step 1: MAP

Input:

```
hadoop is fun
```



Output:

```
(hadoop,1)
(is,1)
(fun,1)
```



Step 2: SHUFFLE

👉 Groups same words:

```
(hadoop → [1])
(is → [1])
(fun → [1])
```



Step 3: REDUCE

👉 Sum values:

```
(hadoop,1)
(is,1)
(fun,1)
```



FROM YOUR OUTPUT (FILE)

From your screenshot  pract11 :

```
fun 1
hadoop 2
is 1
learning 1
mapreduce 1
test 1
```



👉 Meaning:

- "hadoop" appeared 2 times
- others once

MOST IMPORTANT VIVA QUESTIONS

Q1: What is Mapper?

👉 Converts input into key-value pairs

Q2: What is Reducer?

👉 Combines values and gives final output

Q3: What is key-value pair?

👉 Example: (word, count)

? Q4: What is shuffle phase?

👉 Groups similar keys

? Q5: Why Hadoop?

👉 Handles big data efficiently

? Q6: What is HDFS?

👉 Storage system of Hadoop

? Q7: What is Combiner?

👉 Mini reducer (optimization)



COMMON MISTAKES

- ✗ Not explaining map → reduce flow
 - ✗ Forgetting key-value concept
 - ✗ Confusing mapper & reducer
 - ✗ Not mentioning shuffle phase
-



JOURNAL READY

Aim:

To implement WordCount using Hadoop MapReduce.

Conclusion:

Word frequencies were successfully calculated using distributed processing.



HOW TO EXPLAIN IN VIVA

Say this:

- 👉 Mapper converts words into key-value pairs
 - 👉 Shuffle groups similar words
 - 👉 Reducer sums values
 - 👉 Final output shows word frequency
-

FINAL STATUS

Now you have completed:

- ✓ ALL PRACTICALS (1–11)
 - ✓ Python + ML + NLP + Visualization + Hadoop
- 👉 You are **FULLY PREPARED FOR DSBDA PRACTICAL EXAM** 🔥
-

FINAL STEP (DON'T SKIP)

Say:

- 👉 “rapid revision all practicals”

I'll give you:

- 🔥 1-page cheat sheet
- 🔥 Most asked viva questions
- 🔥 Last-minute tricks

(Perfect before entering exam)        Sources

not code just proper commands from pdf to run on ubuntu
cmd

Got it—you don't want Java code, just the **exact Ubuntu/Hadoop commands flow** like in your PDF  pract11 .

Here's the **clean step-by-step you should write/execute** 👉

PRACTICAL 11 – WORDCOUNT (UBUNTU COMMANDS)

✓ 1. Go to Hadoop directory

```
</> Bash
```



```
cd ~
```

✓ 2. Create project folder

```
</> Bash
```



```
mkdir hadoop-test
```

```
cd hadoop-test
```

✓ 3. Create input directory

```
</> Bash
```



```
mkdir input
```

✓ 4. Create input files

```
</> Bash
```



```
echo "hadoop mapreduce test" > input/file1.txt
```

```
echo "learning hadoop is fun" > input/file2.txt
```

✓ 5. Run WordCount (MAIN COMMAND ★★)

```
</> Bash
```



```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount
```

👉 This is the **most important line** (they **WILL** ask this)

⚠️ **If output folder already exists (very common error)**

```
</> Bash
```



```
rm -r output
```

Then run again.

✅ **6. View output**

```
</> Bash
```



```
cat output/part-r-00000
```

 **Expected Output (like your PDF)**

From your file  pract11 :

```
fun 1
hadoop 2
is 1
learning 1
mapreduce 1
test 1
```



 **VIVA READY EXPLANATION (IMPORTANT)**

If examiner asks:

👉 “What did you do?”

Say:

I created input files, ran Hadoop WordCount using the jar command, and the output shows frequency of each word.



SUPER IMPORTANT VIVA QUESTIONS

? Q: Why remove output folder?

👉 Hadoop does not overwrite existing output

? Q: What does this command do?

</> Bash



```
hadoop jar ...
```

👉 Runs MapReduce job

? Q: Where is output stored?

👉 In `output/part-r-00000`



FINAL SHORT VERSION (FOR WRITING IN EXAM)

</> Bash



```
mkdir hadoop-test
```

```
cd hadoop-test
```

```
mkdir input
```

```
echo "hadoop mapreduce test" > input/file1.txt
```

```
echo "learning hadoop is fun" > input/file2.txt
```

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount
```

```
cat output/part-r-00000
```



DONE — THIS IS EXACTLY WHAT THEY EXPECT



If you want last thing:

📖 Sources



pract12.pdf
PDF

Good—this last one is **very simple (Scala + Spark)** and mostly they expect **basic execution + understanding**, not heavy coding.

From your file  `pract12`, the program is just a **simple addition program in Scala**. I'll give you exactly what you need for exam:



PRACTICAL 12 – SCALA (APACHE SPARK)



PROGRAM (AS IN YOUR PDF)

From your file  `pract12` :

</> scala



```
import scala.io.StdIn

object Addition {
  def main(args: Array[String]): Unit = {

    println("Enter first number:")
    val a = StdIn.readInt()

    println("Enter second number:")
    val b = StdIn.readInt()

    val sum = a + b

    println("Sum = " + sum)
  }
}
```



UBUNTU COMMANDS (IMPORTANT)

◆ 1. Open terminal

```
</> Bash
```



```
scala
```

◆ 2. OR run using file

```
</> Bash
```



```
nano Addition.scala
```

👉 paste code → save

◆ 3. Compile

```
</> Bash
```



```
scalac Addition.scala
```

◆ 4. Run

```
</> Bash
```



```
scala Addition
```



OUTPUT (FROM YOUR FILE)

From your PDF  pract12 :

```
Enter first number:
```

```
10
```

```
Enter second number:
```

```
20
```

```
Sum = 30
```





VIVA CONCEPTS (VERY IMPORTANT)

♦ 1. What is Scala?

👉 Answer:

Scala is a programming language used with Apache Spark for big data processing.

♦ 2. What is Apache Spark?

👉 Answer:

Apache Spark is a fast data processing framework used for big data analytics.

♦ 3. Why Scala with Spark?

👉 Because:

- Spark is written in Scala
 - gives best performance
-

♦ 4. What is object in Scala?

👉 Similar to class

👉 Used to define program entry point

♦ 5. What is main()?

👉 Entry point of program

♦ 6. What is val?

👉 Immutable variable (cannot change value)



MOST ASKED VIVA QUESTIONS

? Q1: Difference between Scala & Java?

Scala	Java
functional + OOP	only OOP
shorter code	longer code

? Q2: What is Spark?

👉 Big data processing engine

? Q3: Why Spark over Hadoop?

👉 Faster (in-memory processing)

? Q4: What is RDD? (IMPORTANT)

👉 Resilient Distributed Dataset

👉 Basic data structure in Spark



COMMON MISTAKES

- ✗ Forgetting `object` keyword
- ✗ Wrong file name vs object name
- ✗ Not compiling before running



JOURNAL READY

Aim:

To write and execute a simple Scala program using Apache Spark environment.

Conclusion:

The program successfully performed addition using Scala, demonstrating basic Spark-compatible programming.



FINAL STATUS

Now you have completed:

- ✓ ALL PRACTICALS (1–12)
- ✓ Python + ML + Hadoop + Spark

👉 **YOU ARE 100% READY** 🔥



FINAL STEP (IMPORTANT)

Say:

👉 **“rapid revision all practicals”**

I'll give you:

- 🔥 1-page cheat sheet
- 🔥 Most repeated viva questions
- 🔥 Last-minute memory tricks

(Perfect just before exam 🚀)



Sources